

# AWS GenAI Developer — Professional · Course Map

All 20 tasks, one page. Each task's core idea and the scenario it was taught through.

## Domain 1 — Solution Design

| Task | Title                           | Core idea                                               | Use case                                          |
|------|---------------------------------|---------------------------------------------------------|---------------------------------------------------|
| 1.1  | Architecture & PoC              | How to build the system end to end                      | Bank support assistant, 15 countries, 8 languages |
| 1.2  | FM selection & lifecycle        | How to choose a model and manage it over time           | Healthcare document analysis, 50k docs/day        |
| 1.3  | Data preparation                | What happens to data <i>before</i> it reaches the model | Insurance claims, multimodal (text/image/audio)   |
| 1.4  | Vector stores                   | Where vectors live and how they're maintained           | Law firm, 15M documents, 30 countries             |
| 1.5  | Retrieval design                | How <i>good</i> the retrieval is — recall and relevance | Law firm again, retrieval quality angle           |
| 1.6  | Prompt engineering & governance | How you talk to the model, and how that is governed     | Bank customer service, 50M customers              |

## Domain 2 — Implementation & Integration

| Task | Title                    | Core idea                                           | Use case                                          |
|------|--------------------------|-----------------------------------------------------|---------------------------------------------------|
| 2.1  | Agentic AI & tools       | Autonomy — and the safeguards that make it safe     | Supply chain, 50 plants, multi-agent              |
| 2.2  | Deployment strategies    | Where to run the model; why LLMs differ from ML     | (no narrative — tiered deployment pattern)        |
| 2.3  | Enterprise integration   | Fitting FMs into legacy estates, securely           | Bank, 30 countries, legacy systems, GenAI Gateway |
| 2.4  | FM API integrations      | Talking to the model at API level                   | Healthcare assistant, clinical workflows          |
| 2.5  | App patterns & dev tools | Front end, business systems, developer productivity | Insurance claims platform, 50k claims/month       |

## Domain 3 — Safety, Security & Governance

| Task | Title                   | Core idea                                               | Use case                                    |
|------|-------------------------|---------------------------------------------------------|---------------------------------------------|
| 3.1  | Input/output safety     | Is the <b>content</b> safe? Defense in depth.           | Financial AI assistant, adversarial testing |
| 3.2  | Data security & privacy | Is the <b>data</b> safe? Who sees it, where it goes.    | Healthcare, HIPAA, patient records          |
| 3.3  | Governance & compliance | Can you <b>prove</b> it? Documentation and lineage.     | Financial advisory, SEC/FINRA               |
| 3.4  | Responsible AI          | Is it <b>right for everyone</b> ? Transparent and fair. | Healthcare chatbot, demographic fairness    |

## Domain 4 — Operational Efficiency

| Task | Title                    | Core idea                                   | Use case                                          |
|------|--------------------------|---------------------------------------------|---------------------------------------------------|
| 4.1  | Cost optimisation        | Same work, less money                       | Tiered model usage framework                      |
| 4.2  | Performance optimisation | Lower latency, higher throughput            | Financial market infrastructure                   |
| 4.3  | Monitoring               | GenAI fails in ways ML monitoring can't see | Financial advisory, FM troubleshooting frameworks |

## Domain 5 — Testing, Validation & Troubleshooting

| Task | Title              | Core idea                                  | Use case                                     |
|------|--------------------|--------------------------------------------|----------------------------------------------|
| 5.1  | Evaluation systems | How good is it? Proactive and systematic.  | Financial chatbot, pilot to production       |
| 5.2  | Troubleshooting    | Why is it broken? Reactive and diagnostic. | Healthcare assistant, four reported symptoms |

## The Recurring Threads

| Thread            | Where it runs through the course                                                                                      |
|-------------------|-----------------------------------------------------------------------------------------------------------------------|
| RAG               | 1.1 introduced → 1.4 storage → 1.5 quality → 3.1 as grounding → 2.4 as a fallback tier → 5.1 evaluated → 5.2 debugged |
| Agents            | 1.1 named → 2.1 built and safeguarded → 2.5 rate-limited → 3.4 traced → 4.3 monitored → 5.1 evaluated                 |
| Guardrails        | 1.1 flagged as missing → 1.6 configured → 3.1 as one layer of many → 3.2 for privacy → 3.4 for policy                 |
| Prompts           | 1.1 versioned → 1.6 governed → 2.5 compressed → 4.1 token-optimised → 5.2 debugged                                    |
| Caching           | 2.2 deployment → 4.1 cost (semantic, prompt) → 4.2 latency → 1.4 vector search                                        |
| Human in the loop | 1.2 clinical validation → 1.6 three-tier approval → 2.1 augmentation → 3.3 governance gate → 5.1 expert annotation    |

**The single most useful framing:** Domain 1 designs it, Domain 2 builds it, Domain 3 protects it, Domain 4 runs it efficiently, Domain 5 proves it works. The same services appear in all five — **what changes is the problem they are solving**. That is what the exam tests.